

MegaPose + TurtleBot Real-Time Robotics Demo

2.5-day implementation brief for a career-fair-ready perception-to-action demo

Goal: turn the already-working real-time MegaPose pipeline into a short physical robotics demo that is reliable, visually obvious, technically defensible, and easy to explain in 20-30 seconds.

1. Minimum Viable Demo

A camera observes one of the existing 3D markers. MegaPose estimates its 6-DoF pose in real time. That pose is published into ROS2 and consumed by a simple TurtleBot controller. The robot reacts to the marker position using a small closed-loop control policy.

- **Input:** live RGB camera stream.
- **Perception:** object detection / bounding box + real-time MegaPose.
- **Output:** pose estimate (translation + rotation) and annotated video.
- **ROS2 bridge:** publish estimated pose and optionally confidence/FPS.
- **Robot behavior:** rotate to center target; optionally move forward/backward to maintain distance.
- **Demo artifact:** 20-40 second phone video showing both TurtleBot motion and live pose overlay.

Recommended architecture

```
RGB Camera -> Detector / ROI -> MegaPose -> 6-DoF Pose -> ROS2 Pose Topic -> Simple Controller -> cmd_vel -> TurtleBot
```

Do not add navigation, SLAM, complex planning, or a new detector unless absolutely necessary. The value of this demo is the clean perception-to-action loop, not feature count.

Success criteria

Item	Pass condition
Perception	Stable pose visualization for at least 30-60 seconds
Latency	Responsive enough that robot behavior visibly follows target movement
ROS2	Pose arrives continuously on a dedicated topic
Control	Robot turns toward target reliably; distance control is optional
Safety	Velocity limits and target-loss stop are enforced
Presentation	One clean 20-40 s demo video + one simple architecture image

2. ROS2 Implementation Plan

Suggested nodes

Node	Responsibility	Keep it simple
megapose_node	Read camera frame, obtain bbox/ROI, run MegaPose, draw axes, publish pose	Reuse existing working pipeline
target_controller	Subscribe to pose and generate TurtleBot velocity commands	P-controller only
demo_viewer (optional)	Show annotated frame, FPS, xyz and status	Can stay inside megapose_node

Recommended ROS messages

Prefer standard messages so Codex has less custom-package work:

```
/megapose/object_pose geometry_msgs/PoseStamped /megapose/visible std_msgs/Bool /megapose/fps  
std_msgs/Float32 # optional /cmd_vel geometry_msgs/Twist
```

Coordinate convention

Pick one frame and document it. The easiest first version is to publish the pose in the camera optical frame. For the control demo, you only need the components that map cleanly to target left/right and depth. Do not spend hours building a full tf tree unless your current setup already supports it.

Minimal controller

Assuming the target pose gives horizontal offset x and forward depth z in your chosen camera convention:

```
if target_not_visible: linear_x = 0 angular_z = 0 else: angular_z = clamp(-K_yaw * x_error,  
-w_max, w_max) linear_x = 0 # Optional after yaw-only is stable: distance_error = z -  
desired_distance linear_x = clamp(K_dist * distance_error, -v_max, v_max)
```

First milestone: yaw-only tracking. It is enough for the fair. Add distance control only if the yaw behavior is already boringly reliable.

Safety requirements

- Immediately publish zero velocity when target is lost for N consecutive frames.
- Clamp angular and linear speed to conservative values.
- Add a keyboard kill switch or ROS2 service/topic that forces stop.
- Never let stale pose messages drive the robot; reject old timestamps.
- Record the demo in an uncluttered area with enough stopping distance.

3. 2.5-Day Execution Schedule

Window	Objective	Definition of done
Day 1 - first half	Stabilize MegaPose live pipeline	Annotated live stream, pose + FPS, no ROS control
Day 1 - second half	Wrap/publish pose in ROS2	ros2 topic echo shows stable PoseStamped
Day 2 - first half	Yaw-only TurtleBot controller	Robot rotates to face target and stops on target loss
Day 2 - second half	Optional distance control + cleanup	Approach/retreat works without oscillation OR skip it
Day 3 / before travel	Record and package demo	20-40 s video, README/diagram, code committed
Only if ahead	Qwen mini-demo	VLM produces a constrained high-level command, not direct motor output

Cut features in this order if time slips

- Qwen/VLM integration.
- Distance control.
- Fancy UI / web dashboard.
- Full tf frame integration.
- Automatic object discovery or retraining.

Do **not** cut the reliable live pose overlay, ROS2 pose publication, target-loss stop, or the recorded demo.

4. Optional Qwen Layer - only after the core demo works

Use Qwen for semantic intent, not low-level control. The clean architecture is:

```
User text / image -> Qwen VLM -> constrained intent JSON -> deterministic ROS2 behavior | +-> e.g.
{"behavior": "track", "target": "marker", "distance_m": 0.8}
```

This lets you explain that the foundation model handles semantics while geometry, control and safety remain deterministic. That is a stronger robotics story than allowing a VLM to directly generate velocity commands.

Potential 10-minute extension

- Prompt: "Track the marker and stay about 0.8 m away."
- Qwen converts it to a tiny validated schema.
- Controller switches from stop to tracking mode and uses MegaPose for actual geometry.

5. Codex Handoff Prompt

Paste the following into the coding agent together with the existing MegaPose codebase:

```
I already have a working real-time MegaPose pipeline. Do not rewrite or replace the core inference unless required. Goal: turn it into a ROS2 Jazzy TurtleBot demo in which a camera observes a known object/3D marker, MegaPose estimates its 6-DoF pose in real time, and a simple ROS2 controller makes the TurtleBot rotate toward the target. Distance control is optional and should only be added after yaw tracking is stable. Implement incrementally: 1. Wrap the existing real-time MegaPose loop in a ROS2 node. 2. Publish geometry_msgs/PoseStamped on /megapose/object_pose. 3. Publish target visibility, or provide an equivalent robust target-lost signal. 4. Preserve the live visualization with bbox/axes and add xyz + FPS if easy. 5. Create a separate target_controller node that subscribes to the pose and publishes geometry_msgs/Twist to /cmd_vel. 6. Start with yaw-only proportional control. Add velocity clamps, stale-pose rejection and immediate stop on target loss. 7. Keep constants such as Kp, max velocity and desired distance as ROS parameters. 8. Keep changes minimal and modular. Do not add Nav2, SLAM, MoveIt, custom messages, or unnecessary dependencies. 9. Provide launch instructions and a short test checklist. Before editing, inspect my current project structure and identify the smallest integration path. Make one functional change at a time and preserve any working MegaPose behavior.
```

6. Career-Fair Presentation

20-second explanation: "I converted a research-grade 6-DoF pose estimator into a real-time pipeline and integrated it with ROS2. The TurtleBot consumes the estimated pose and closes the perception-control loop by reacting to the target in real time. I kept the low-level control deterministic, and I am also experimenting with VLMs for higher-level semantic commands."

What to have on your phone/laptop

- 20-40 second physical demo video saved locally.
- One architecture diagram: Camera -> MegaPose -> ROS2 -> Controller -> TurtleBot.
- GitHub/README with 3-5 bullets: problem, architecture, what you implemented, measured FPS/latency if available.
- A second fallback video showing only the stable live MegaPose pose overlay.

After the fair: reuse the same real-time pipeline to collect controlled physical results on the existing markers, then transition into the conference-paper experiments rather than abandoning the code as a one-off demo.